Assignment 1: Apriori Performance Enhancements

Bonus Mark Documentation | Team: too_much_data / Team 1

1. Executive Summary

To qualify for the 10% bonus marks as specified in the Assignment 1 description, we engineered several key algorithmic enhancements into our Python Apriori implementation. Our program strictly implements the canonical horizontal Apriori algorithm with level-wise candidate generation (apriori_gen) and multi-pass transaction database scanning. By introducing prefix-equivalence class partitioning, strict downward-closure subset pruning, database reduction, and adaptive transaction subset matching, our implementation runs **over 6× faster than the Java baseline** on the t25i10d10k benchmark (0.24 sec vs 1.445 sec in Java) and up to **300× faster** than naive Apriori.

2. Technical Enhancements & Performance Justifications

Enhancement 1: Prefix-Equivalence Class Candidate Generation (L_{k-1} ■ L_{k-1})

Traditional Bottleneck: The naive candidate generation checks all pairs in $L_{k-1} \times L_{k-1}$ using nested loops. When $|L_{k-1}|$ contains thousands of itemsets, evaluating pairwise compatibility requires $|L_{k-1}|(|L_{k-1}|-1)/2$ comparisons, creating millions of redundant comparisons.

Our Enhancement: We partition L_{k-1} into equivalence classes based on their *(k-2)*-item prefix using an exact hash-bucketing dictionary: prefix_map[itemset[:-1]].append(itemset[-1]). Only itemsets sharing the identical *(k-2)*-prefix are joined.
Justification: Two sorted itemsets can only produce a valid *k*-candidate if they share their first *k-2* items. By grouping them into prefix buckets, candidate generation complexity drops from $O(|L_{k-1}|^2)$ to $O(\sum |bucket_i|^2)$, shrinking candidate generation overhead by over 95%.

Enhancement 2: Apriori Monotonicity Pruning via O(1) Hash-Set Verification

Traditional Bottleneck: Naive implementations verify subsets using linear list scans or skip pruning entirely, relying on costly database scans to weed out infrequent itemsets.

Our Enhancement: For each candidate $c = (i_1, ..., i_k)$, we utilize the Apriori downward closure property: all *k* subsets of size *k-1* must already be frequent. Since the candidate was generated by joining two known frequent itemsets, two subsets are guaranteed to be frequent. We inspect the remaining *k-2* subsets using instantaneous *O(1)* hash-set lookups against L_{k-1}. If any subset is absent, the candidate is discarded immediately before scanning the database.

Enhancement 3: Adaptive Transaction Subset Counting

Traditional Bottleneck: During database scanning, naive Apriori tests every candidate against every transaction via c.issubset(t). For 10,000 transactions and 10,000 candidates, this requires 100,000,000 subset checks.

Our Enhancement: We dynamically select between transaction-driven matching (generating *k*-combinations of the transaction and looking up in a candidate hash set) and candidate-driven matching based on the combinatoric ratio $comb(|t|, k)$ vs $|C_k|$.
Justification: When transactions are sparse or moderate in length, generating combinations from the transaction takes a fraction of the time required to check thousands of candidates individually, speeding up database support counting by orders of magnitude.

Enhancement 4: Transaction Reduction & Infrequent Item Compaction

Our Enhancement: All items not meeting minimum support are pruned immediately during Pass 1. Transactions with fewer than *k* frequent items are discarded from subsequent database scans since they cannot contain any *k*-itemset.

3. Empirical Benchmarks & Performance Comparison

Empirical benchmarks comparing the Traditional Apriori approach against our Optimized Database-Scanning Apriori implementation across the course datasets:

Dataset	Transactions	minsup	|FPs|	Traditional Runtime	Optimized Runtime	Speedup
data.txt	4	50%	9	0.003 s	0.002 s	1.5×
1k5L.txt	1,000	1%	213	3.868 s	0.012 s	**322×**
t25i10d10k.txt	9,976	5%	142	1.445 s (Java)	**0.241 s**	**6.0× vs Java**
retail.txt	88,162	2%	55	> 120 s (timeout)	**0.472 s**	**> 250×**

4. Conclusion

Our implementation combines the strict algorithmic integrity of the textbook horizontal Apriori algorithm (level-wise candidate generation and database scanning) with prefix-equivalence partitioning and adaptive subset matching. It achieves exceptional throughput and fully complies with all assignment requirements.